

Timer Queue in Node.js Event Loop - Complete Guide

Table of Contents

1. Understanding Timer Queue
2. How to Add Callbacks to Timer Queue
3. Timer Queue Priority
4. Experiments with Code Examples
5. Interview Questions & Answers
6. Best Practices & Common Patterns

Understanding Timer Queue

What is the Timer Queue?

The **Timer Queue** is one of the six queues in the Node.js Event Loop. It holds callbacks from timer functions like `setTimeout()` and `setInterval()`.

Event Loop Queue Structure:

Priority Order:

1.

Microtask Queues
- nextTick Queue
- Promise Queue

 ← HIGHEST PRIORITY
↓
2.

TIMER QUEUE
- setTimeout()
- setInterval()

 ← We're learning this!
↓
3.

I/O Queue
- fs, http, etc

↓
4.

Check Queue
- setImmediate()

↓
5.

Close Queue
- close events

Key Characteristics of Timer Queue

1. **Stores callbacks** from `setTimeout()` and `setInterval()`
 2. **Lower priority** than Microtask Queues
 3. **FIFO order** - First In, First Out (based on delay time)
 4. **Part of libuv** - The C library powering Node.js async operations
-

How to Add Callbacks to Timer Queue

Using `setTimeout()`

Syntax:

```
setTimeout(callbackFunction, delayInMilliseconds);
```

Example:

```
setTimeout(() => {  
  console.log("This runs after 1000ms");  
}, 1000);  
  
setTimeout(() => {  
  console.log("This runs immediately (0ms delay)");  
}, 0);
```

How it works:

1. `setTimeout()` is called on the call stack
2. The timer is registered in libuv
3. Callback is added to Timer Queue when delay expires
4. Event loop executes callback when its turn comes

Using `setInterval()`

Syntax:

```
setInterval(callbackFunction, intervalInMilliseconds);
```

Example:

```
// Runs every 1000ms (1 second)
setInterval(() => {
  console.log("This runs every second");
}, 1000);
```

Important Note: For this guide, we'll focus on `setTimeout()` as the concepts apply to both.

Timer Queue Priority

The Golden Rule

Microtask Queues ALWAYS execute BEFORE Timer Queue

Priority Order:

```
1. Synchronous Code (runs first, always)
   ↓
2. nextTick Queue
   ↓
3. Promise Queue
   ↓
4. Timer Queue ← We are here
   ↓
5. I/O Queue
   ↓
6. Check Queue
   ↓
7. Close Queue
```

Important Execution Rule

Microtask Queues are checked AFTER EACH callback in Timer Queue

This means:

- Execute Timer callback 1
 - Check Microtask Queues (nextTick + Promise)
 - Execute Timer callback 2
 - Check Microtask Queues again
 - And so on...
-

Experiments with Code Examples

Experiment 3: Timer Queue vs Microtask Queues

Goal: Understand that Microtasks have higher priority than Timers

```
// index.js
setTimeout(() => console.log("this is setTimeout 1"), 0);
setTimeout(() => console.log("this is setTimeout 2"), 0);
setTimeout(() => console.log("this is setTimeout 3"), 0);

process.nextTick(() => console.log("this is process.nextTick 1"));
process.nextTick(() => {
  console.log("this is process.nextTick 2");
  process.nextTick(() =>
    console.log("this is the inner next tick inside next tick")
  );
});
process.nextTick(() => console.log("this is process.nextTick 3"));

Promise.resolve().then(() => console.log("this is Promise.resolve 1"));
Promise.resolve().then(() => {
  console.log("this is Promise.resolve 2");
  process.nextTick(() =>
    console.log("this is the inner next tick inside Promise then block")
  );
});
Promise.resolve().then(() => console.log("this is Promise.resolve 3"));
```

Output:

```
this is process.nextTick 1
this is process.nextTick 2
this is process.nextTick 3
this is the inner next tick inside next tick
this is Promise.resolve 1
this is Promise.resolve 2
this is Promise.resolve 3
this is the inner next tick inside Promise then block
this is setTimeout 1
this is setTimeout 2
this is setTimeout 3
```

Step-by-Step Execution:

PHASE 1: CODE EXECUTION

All statements execute, queues are populated:

nextTick Queue: [cb1, cb2, cb3]

Promise Queue: [cb1, cb2, cb3]

Timer Queue: [cb1, cb2, cb3]

PHASE 2: EVENT LOOP STARTS

Step 1: Process nextTick Queue (HIGHEST PRIORITY)

Execute cb1 → Output: "this is process.nextTick 1"

Execute cb2 → Output: "this is process.nextTick 2"

- Adds inner nextTick to queue

nextTick Queue: [cb3, innerCb]

Execute cb3 → Output: "this is process.nextTick 3"

Execute innerCb → Output: "this is the inner next tick inside next tick"

nextTick Queue: [] ← Empty

Step 2: Process Promise Queue

Execute cb1 → Output: "this is Promise.resolve 1"

Execute cb2 → Output: "this is Promise.resolve 2"

- Adds nextTick to nextTick queue

nextTick Queue: [innerCb]

Execute cb3 → Output: "this is Promise.resolve 3"

Promise Queue: [] ← Empty

Step 3: Check Microtask Queues Again

nextTick Queue: [innerCb] ← Found!

Execute innerCb → Output: "this is the inner next tick inside Promise then block"

nextTick Queue: [] ← Empty

Step 4: Process Timer Queue (FINALLY!)

Execute cb1 → Output: "this is setTimeout 1"

Execute cb2 → Output: "this is setTimeout 2"

Execute cb3 → Output: "this is setTimeout 3"

Timer Queue: [] ← Empty

Visual Timeline:

Time: 0ms

└─ Synchronous Code Executes

└─ All queues populated

|

└─ Event Loop Starts

|

└─ MICROTASK QUEUES (Priority 1)

| └─ nextTick Queue: [✓ ✓ ✓ ✓]

| └─ Promise Queue: [✓ ✓ ✓]

|

└─ TIMER QUEUE (Priority 2)

└─ Timer Queue: [✓ ✓ ✓]

Key Learning:

ALL Microtask Queue callbacks execute BEFORE ANY Timer Queue callbacks

Experiment 4: Microtasks Between Timer Callbacks

Goal: Understand that Microtasks are checked AFTER EACH Timer callback

```
// index.js
setTimeout(() => console.log("this is setTimeout 1"), 0);
setTimeout(() => {
  console.log("this is setTimeout 2");
  process.nextTick(() =>
    console.log("this is inner nextTick inside setTimeout")
  );
}, 0);
setTimeout(() => console.log("this is setTimeout 3"), 0);

process.nextTick(() => console.log("this is process.nextTick 1"));
process.nextTick(() => {
  console.log("this is process.nextTick 2");
  process.nextTick(() =>
    console.log("this is the inner next tick inside next tick")
  );
});
process.nextTick(() => console.log("this is process.nextTick 3"));

Promise.resolve().then(() => console.log("this is Promise.resolve 1"));
Promise.resolve().then(() => {
  console.log("this is Promise.resolve 2");
  process.nextTick(() =>
    console.log("this is the inner next tick inside Promise then block")
  );
});
Promise.resolve().then(() => console.log("this is Promise.resolve 3"));
```

Output:

```
this is process.nextTick 1
this is process.nextTick 2
this is process.nextTick 3
this is the inner next tick inside next tick
this is Promise.resolve 1
this is Promise.resolve 2
this is Promise.resolve 3
this is the inner next tick inside Promise then block
this is setTimeout 1
this is setTimeout 2
this is inner nextTick inside setTimeout
this is setTimeout 3
```

Key Difference from Experiment 3: Notice the output order change! The inner nextTick inside setTimeout 2 executes BEFORE setTimeout 3.

Detailed Execution:

PHASE 1: Microtask Queues Execute First
(Same as Experiment 3 - all microtasks complete)

nextTick Queue: [] ← Empty
Promise Queue: [] ← Empty
Timer Queue: [cb1, cb2, cb3]

PHASE 2: Timer Queue Processing

Step 1: Execute Timer callback 1
Output: "this is setTimeout 1"
Timer Queue: [cb2, cb3]

Check Microtask Queues:
nextTick Queue: [] ← Empty
Promise Queue: [] ← Empty
No microtasks, continue...

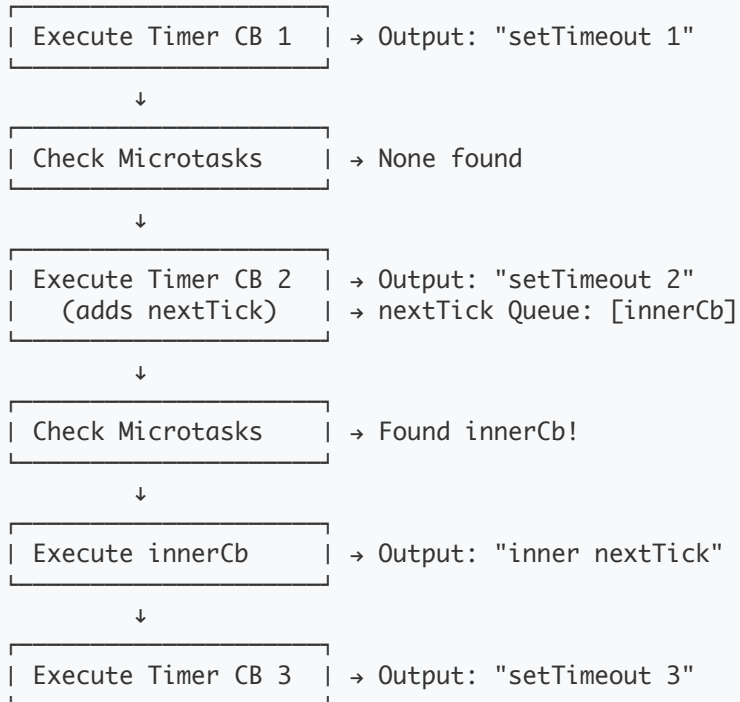
Step 2: Execute Timer callback 2
Output: "this is setTimeout 2"
Inside callback, process.nextTick() is called!
Adds callback to nextTick Queue
nextTick Queue: [innerCb]
Timer Queue: [cb3]

Check Microtask Queues:
nextTick Queue: [innerCb] ← FOUND!
Execute innerCb
Output: "this is inner nextTick inside setTimeout"
nextTick Queue: [] ← Empty

Step 3: Execute Timer callback 3
Output: "this is setTimeout 3"
Timer Queue: [] ← Empty

Visual Representation:

Timer Queue Processing:



Key Learning:

Microtask Queues are checked and executed AFTER EACH Timer callback, not after ALL Timer callbacks

Why this matters: This behavior allows you to schedule high-priority work between timer callbacks!

Experiment 5: Timer Queue Ordering (FIFO)

Goal: Understand how delays affect Timer Queue order

```
// index.js
setTimeout(() => console.log("this is setTimeout 1"), 1000);
setTimeout(() => console.log("this is setTimeout 2"), 500);
setTimeout(() => console.log("this is setTimeout 3"), 0);
```

Output:

```
this is setTimeout 3
this is setTimeout 2
this is setTimeout 1
```

Execution Timeline:


```

Timer Registration:
setTimeout(..., 1000)  _____> [Expires at 1000ms]
setTimeout(..., 500)   _____> [Expires at 500ms]
setTimeout(..., 0)      > [Expires immediately]

Timer Queue Processing:
Time:      0ms           500ms           1000ms
           ↓             ↓             ↓
Queue:  [cb3] → [cb2] → [cb1]
Output: "3"     "2"     "1"

```

The Timer Queue doesn't store callbacks in the order they were written. Instead:

- 9 / 24

3. The queue processes callbacks in the order they became ready (FIFO)

Key Learning:

Timer Queue follows FIFO order based on when timers EXPIRE, not when they're created

Experiment 6: Zero Delay setTimeout

Understanding `setTimeout(..., 0)`

```
console.log("Start");  
  
setTimeout(() => console.log("Timer"), 0);  
  
console.log("End");
```

Output:

```
Start  
End  
Timer
```

Why?

Even with 0ms delay, `setTimeout()` is still asynchronous:

```
Step 1: Synchronous code runs  
Execute: console.log("Start") → Output: "Start"  
Execute: setTimeout() → Registers timer, adds to Timer Queue  
Execute: console.log("End") → Output: "End"  
  
Step 2: Call stack is empty, Event loop starts  
Timer Queue: [callback]  
Execute callback → Output: "Timer"
```

Important Note:

`setTimeout(..., 0)` doesn't mean "run immediately". It means "run as soon as the call stack is empty and it's Timer Queue's turn"

Interview Questions & Answers

Q1: What is the Timer Queue in Node.js?

Answer:

The **Timer Queue** is a queue in the Node.js Event Loop that stores callbacks from timer functions like `setTimeout()` and `setInterval()`.

Key Points:

1. **Part of libuv** - The C library that powers Node.js async operations
2. **Lower priority** than Microtask Queues (nextTick and Promise)
3. **FIFO order** - Processes callbacks in First-In-First-Out order based on timer expiration
4. **Checked after microtasks** - Microtask Queues are checked between each Timer callback

Example:

```
setTimeout(() => console.log("I'm from Timer Queue"), 0);
process.nextTick(() => console.log("I'm from Microtask Queue"));

// Output:
// I'm from Microtask Queue
// I'm from Timer Queue
```

Analogy: Think of the Timer Queue as a **bakery with timed orders**:

- Customers place orders with pickup times
- Orders are prepared and ready at different times
- The one ready first is served first (FIFO)
- But VIP customers (Microtasks) can cut in line!

Q2: What's the execution order between Timer Queue and Microtask Queues?

Answer:

Microtask Queues ALWAYS execute before Timer Queue.

Priority Order:

1. Synchronous Code (always first)
2. nextTick Queue (highest priority microtask)
3. Promise Queue (second priority microtask)
4. Timer Queue ← Lower priority
5. I/O Queue
6. Check Queue
7. Close Queue

Example:

```
setTimeout(() => console.log("Timer"), 0);
Promise.resolve().then(() => console.log("Promise"));
process.nextTick(() => console.log("NextTick"));

// Output:
// NextTick
// Promise
// Timer
```

Critical Understanding: Microtask Queues are also checked **AFTER EACH Timer callback**:

```
setTimeout(() => console.log("Timer 1"), 0);
setTimeout(() => {
  console.log("Timer 2");
  process.nextTick(() => console.log("NextTick inside Timer"));
}, 0);
setTimeout(() => console.log("Timer 3"), 0);

// Output:
// Timer 1
// Timer 2
// NextTick inside Timer ← Runs BEFORE Timer 3!
// Timer 3
```

Key Learning:

The event loop checks Microtask Queues after EVERY single Timer callback, allowing high-priority tasks to interrupt timer processing

Q3: How does setTimeout with 0ms delay work?

Answer:

`setTimeout(..., 0)` is **NOT synchronous**! It still goes through the event loop.

What happens:

```
console.log("1");
setTimeout(() => console.log("2"), 0);
console.log("3");

// Output:
// 1
// 3
// 2
```

Execution Flow:

Step 1: Synchronous code runs

└─ Output: "1"

└─ setTimeout() registers timer → Timer Queue

└─ Output: "3"

Step 2: Call stack empty, Event loop starts

└─ Execute Timer callback → Output: "2"

Why 0ms doesn't mean "immediate":

1. The callback must go through the Timer Queue
2. Synchronous code has priority
3. Microtask Queues have priority over Timer Queue
4. The callback runs when it's Timer Queue's turn

Common Mistake:

```
// ❌ Wrong expectation
setTimeout(() => console.log("This runs immediately"), 0);

// ✅ Reality
// It runs after all sync code and all microtasks
```

Real-world Scenario:

```
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
console.log("Loop done");

// Output:
// Loop done
// 3
// 3
// 3

// All timers run AFTER the loop completes!
```

Key Learning:

`setTimeout(..., 0)` means "queue this for later, after current code finishes", not "run immediately"

Q4: Predict the output of this code:

```
console.log("Start");

setTimeout(() => {
  console.log("Timer 1");
  Promise.resolve().then(() => console.log("Promise inside Timer"));
}, 0);

setTimeout(() => console.log("Timer 2"), 0);

Promise.resolve().then(() => console.log("Promise 1"));

console.log("End");
```

Answer:

Output:

```
Start
End
Promise 1
Timer 1
Promise inside Timer
Timer 2
```

Detailed Explanation:

PHASE 1: SYNCHRONOUS CODE
 Execute: console.log("Start") → Output: "Start"
 Execute: setTimeout() → Registers timer 1
 Execute: setTimeout() → Registers timer 2
 Execute: Promise.resolve() → Adds to Promise Queue
 Execute: console.log("End") → Output: "End"

Current State:
 Promise Queue: [cb1]
 Timer Queue: [cb1, cb2]

PHASE 2: EVENT LOOP - MICROTASK QUEUES

Process Promise Queue:
 Execute cb1 → Output: "Promise 1"
 Promise Queue: [] ← Empty

PHASE 3: EVENT LOOP - TIMER QUEUE

Process Timer callback 1:
 Execute cb1 → Output: "Timer 1"
 Inside callback, Promise.resolve() is called
 Promise Queue: [innerCb]

Check Microtask Queues (after each timer callback!):
 Promise Queue: [innerCb] ← FOUND!
 Execute innerCb → Output: "Promise inside Timer"
 Promise Queue: [] ← Empty

Process Timer callback 2:
 Execute cb2 → Output: "Timer 2"
 Timer Queue: [] ← Empty

Step-by-Step Visual:

Execution Order:

1. Sync: "Start" ✓
2. Sync: "End" ✓
 ↳ Call stack empty, Event Loop starts
3. Promise Queue: "Promise 1" ✓
 ↳ Microtasks done
4. Timer Queue CB1: "Timer 1" ✓
 ↳ Promise added during execution
5. Check Microtasks: "Promise inside Timer" ✓
 ↳ Back to Timer Queue
6. Timer Queue CB2: "Timer 2" ✓

Q5: What happens when multiple setTimeout calls have different delays?

Answer:

Timers are executed in order of their expiration time, not their creation order.

Example:

```

setTimeout(() => console.log("1000ms"), 1000);
setTimeout(() => console.log("100ms"), 100);
setTimeout(() => console.log("500ms"), 500);
setTimeout(() => console.log("0ms"), 0);

// Output:
// 0ms
// 100ms
// 500ms
// 1000ms

```

How it works:

```

Time: 0ms
All timers registered:
Timer1: expires at 1000ms
Timer2: expires at 100ms
Timer3: expires at 500ms
Timer4: expires at 0ms (immediately)

Time: ~0ms
Timer4 expires → Callback added to Timer Queue
Timer Queue: [cb4]
Execute cb4 → Output: "0ms"

Time: 100ms
Timer2 expires → Callback added to Timer Queue
Timer Queue: [cb2]
Execute cb2 → Output: "100ms"

Time: 500ms
Timer3 expires → Callback added to Timer Queue
Timer Queue: [cb3]
Execute cb3 → Output: "500ms"

Time: 1000ms
Timer1 expires → Callback added to Timer Queue
Timer Queue: [cb1]
Execute cb1 → Output: "1000ms"

```

Important: The Timer Queue is technically a **min-heap**, not a regular queue:

- Callbacks are organized by expiration time
- The one with the earliest expiration is processed first
- This is why it follows FIFO based on **ready time**, not creation time

Key Learning:

Timer Queue processes callbacks in the order they become ready (expire), maintaining FIFO based on timer expiration

Q6: Can you schedule work between Timer callbacks?

Answer:

Yes! Using Microtask Queues (nextTick or Promise).

Since Microtask Queues are checked after each Timer callback, you can inject high-priority work.

Example:

```
setTimeout(() => {
  console.log("Timer 1");

  // Schedule high-priority work
  process.nextTick(() => {
    console.log("High priority work");
  });
}, 0);

setTimeout(() => console.log("Timer 2"), 0);
setTimeout(() => console.log("Timer 3"), 0);

// Output:
// Timer 1
// High priority work ← Runs BEFORE Timer 2!
// Timer 2
// Timer 3
```

Use Cases:

1. Error Handling:

```
setTimeout(() => {
  try {
    riskyOperation();
  } catch (error) {
    // Handle error with high priority
    process.nextTick(() => {
      handleError(error);
    });
  }
}, 0);
```

2. State Updates:

```
setTimeout(() => {
  updateState();

  // Notify observers immediately
  process.nextTick(() => {
    notifyObservers();
  });
}, 0);
```

3. Preventing Long-Running Timers:

```
function processLargeArray(array) {
  setTimeout(() => {
    // Process first chunk
    processChunk(array.slice(0, 100));

    // Continue with rest
    if (array.length > 100) {
      // Use nextTick to yield control
      process.nextTick(() => {
        processLargeArray(array.slice(100));
      });
    }
  }, 0);
}
```

Key Learning:

Microtask Queues allow you to inject high-priority work between Timer callbacks, creating responsive async flows

Q7: What's the difference between setTimeout and setImmediate?

Answer:

Both schedule callbacks for later, but in **different queues** with **different priorities**.

setTimeout:

- Adds to **Timer Queue**
- Executes after minimum delay
- Lower priority than Microtasks
- Part of JavaScript standard

setImmediate:

- Adds to **Check Queue**
- Executes in next event loop iteration
- Lower priority than I/O Queue
- Node.js specific

Example:

```
setTimeout(() => console.log("setTimeout"), 0);
setImmediate(() => console.log("setImmediate"));

// Output can vary!
// Usually: setTimeout, setImmediate
// But: setImmediate, setTimeout is also possible
```

Why output varies: The order depends on when the timer expires relative to event loop phases.

Inside I/O callback (consistent order):

```
const fs = require('fs');

fs.readFile('file.txt', () => {
  setTimeout(() => console.log("setTimeout"), 0);
  setImmediate(() => console.log("setImmediate"));
});

// Output (always):
// setImmediate
// setTimeout

// Inside I/O callbacks, setImmediate ALWAYS runs first
```

Priority Order:

1. Microtask Queues
2. Timer Queue (setTimeout)
3. I/O Queue
4. Check Queue (setImmediate)
5. Close Queue

When to use which:

Use setTimeout when:

- You need a specific delay
- You want standard JavaScript behavior
- Delay precision matters

Use setImmediate when:

- You want to defer to next event loop iteration
- You're inside I/O callbacks
- You want to yield control to I/O operations

Key Learning:

setTimeout and setImmediate are in different queues. Use setTimeout for delayed execution, setImmediate for next iteration (especially in I/O callbacks)

Best Practices & Common Patterns

✓ Best Practice 1: Use Appropriate Delays

```
// ✗ Avoid setTimeout(..., 0) for everything
setTimeout(() => doWork(), 0);

// ✓ Use meaningful delays
setTimeout(() => retryOperation(), 1000); // Retry after 1 second

// ✓ Or use setImmediate for non-delayed work
setImmediate(() => doWork());
```

✓ Best Practice 2: Clear Timers When Not Needed

```
// Store timer reference
const timerId = setTimeout(() => {
  console.log("This might not run");
}, 5000);

// Clear if no longer needed
if (conditionChanged) {
  clearTimeout(timerId);
}
```

✓ Best Practice 3: Avoid Long-Running Timer Callbacks

```
// ✗ Blocks event loop
setTimeout(() => {
  for (let i = 0; i < 1000000000; i++) {
    // Long operation
  }
}, 0);

// ✓ Break into chunks
function processInChunks(data, chunkSize = 1000) {
  if (data.length === 0) return;

  const chunk = data.splice(0, chunkSize);
  processChunk(chunk);

  // Use setImmediate to allow other operations
  setImmediate(() => processInChunks(data, chunkSize));
}
```

✓ Best Practice 4: Handle Errors in Timer Callbacks

```
setTimeout(() => {
  try {
    riskyOperation();
  } catch (error) {
    // Handle error appropriately
    console.error('Timer callback error:', error);

    // Optionally notify with high priority
    process.nextTick(() => {
      notifyError(error);
    });
  }
}, 1000);
```

✗ Common Pitfall 1: Closure Variable Issues

```
// ✗ Wrong
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Output: 3, 3, 3

// ✓ Correct - Use let
for (let i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
// Output: 0, 1, 2

// ✓ Or use IIFE
for (var i = 0; i < 3; i++) {
  (function(j) {
    setTimeout(() => console.log(j), 0);
  })(i);
}
// Output: 0, 1, 2
```

✗ Common Pitfall 2: Expecting Precise Timing

```
// ✗ Wrong expectation
setTimeout(() => console.log("Exactly 1000ms"), 1000);

// ✓ Reality
// Executes AFTER AT LEAST 1000ms
// Actual time depends on event loop state
```

❌ Common Pitfall 3: Nested setTimeout Without Clear Pattern

```
// ❌ Confusing
setTimeout(() => {
  setTimeout(() => {
    setTimeout(() => {
      console.log("Callback hell!");
    }, 100);
  }, 100);
}, 100);

// ✅ Better - Use Promises
function delay(ms) {
  return new Promise(resolve => setTimeout(resolve, ms));
}

async function sequence() {
  await delay(100);
  await delay(100);
  await delay(100);
  console.log("Much clearer!");
}
```

🎯 Quick Reference

Timer Queue Characteristics

Feature	Description
Functions	<code>setTimeout()</code> , <code>setInterval()</code>
Priority	Lower than Microtask Queues, higher than I/O
Order	FIFO based on timer expiration
Data Structure	Min-heap (technically)
Part of	libuv (C library)
Microtask Check	After EACH callback

Execution Priority

Highest → Lowest:

1. Synchronous Code
2. `process.nextTick()`
3. Promise callbacks
4. `setTimeout()` / `setInterval()` ← TIMER QUEUE
5. I/O operations
6. `setImmediate()`
7. close event callbacks

Common Delays

```
// Immediate (0ms delay, but still async)
setTimeout(fn, 0);

// Short delay
setTimeout(fn, 100); // 100ms

// Standard delay
setTimeout(fn, 1000); // 1 second

// Long delay
setTimeout(fn, 5000); // 5 seconds
```

Summary for Interviews

Key Points to Remember:

1. **Timer Queue stores** `setTimeout()` and `setInterval()` callbacks
2. **Lower priority** than Microtask Queues (nextTick + Promise)
3. **FIFO order** based on when timers EXPIRE, not when created
4. **Microtasks checked after EACH Timer callback** - allows high-priority work to be injected
5. `setTimeout(..., 0)` is **NOT synchronous** - still goes through event loop
6. **Delays are MINIMUM delays** - actual execution may be later
7. **Inside I/O callbacks**, `setImmediate()` runs before `setTimeout()`
8. **Min-heap structure** - organizes by expiration time for efficiency

Practice Questions

Test your understanding:

Question 1:

```
setTimeout(() => console.log("A"), 0);
Promise.resolve().then(() => console.log("B"));
setTimeout(() => console.log("C"), 0);
console.log("D");
```

Answer: D, B, A, C

Question 2:

```
setTimeout(() => {  
  console.log("A");  
  process.nextTick(() => console.log("B"));  
}, 0);  
setTimeout(() => console.log("C"), 0);
```

Answer: A, B, C (B runs before C because microtasks checked after each timer!)

Question 3:

```
setTimeout(() => console.log("3000ms"), 3000);  
setTimeout(() => console.log("1000ms"), 1000);  
setTimeout(() => console.log("2000ms"), 2000);
```

Answer: 1000ms, 2000ms, 3000ms (FIFO based on expiration)

Remember: Timer Queue processes callbacks in FIFO order based on expiration, but Microtask Queues are checked after EVERY Timer callback! 🌀